RAI SCHOOL OF ENGINEERING

RAI UNIVERSITY

LAB MANUAL

Artificial Intelligence Essential

Name: _____

Enrollment No.: _____

Programme / Semester: _____

Section / Batch: _____

Submitted To: _____

LIST OF EXPERIMENTS

Artificial Intelligence Essential

Sr. No	Name of Experiment	Date	Page	Grade	Signature
1	Write a program to implement Tic-Tac-Toe game problem.		3		
2	Write a program to implement BFS (for 8 puzzle problem or Water Jug problem or any AI search problem).		6		
3	Write a program to implement DFS (for 8 puzzle problem or Water Jug problem or any AI search problem)		8		
4	Write a program to implement Single Player Game (Using any Heuristic Function)		10		
5	Write a program to Implement A* Algorithm.		12		
6	Write a program to implement mini-max algorithm for any game development.		14		
7	Assume given a set of facts of the form father(name1,name2) (name1 is the father of name2).		16		
8	Define a predicate brother(X,Y) which holds iff X and Y are brothers. Define a predicate cousin(X,Y) which holds iff X and Y are cousins. Define a predicate grandson(X,Y) which holds iff X is a grandson of Y. Define a predicate descendent(X,Y) which holds iff X is a descendent of Y. Consider the following genealogical tree: father(a,b). father(a,c). father(b,d). father(b,e). father(c,f). Say which answers, and in which order, are generated by your definitions for the following queries in Prolog:		17		
9	Write a program to solve Tower of Hanoi problem using Prolog..		19		
10	Write a program to solve N-Queens problem using Prolog.		20		
11	Write a program to solve 8 puzzle problem using Prolog.		21		
12	Write a program to solve travelling salesman problem using Prolog.		23		

Experiment 1

Language: Python 3 **Date:** _____

Problem Statement

Write a program to implement Tic-Tac-Toe game problem.

Aim

To implement the Tic-Tac-Toe game (Human vs Computer).

Theory

The board is a list of 9 cells. The human plays X and the computer plays O. The computer uses a simple rule-based strategy:

1. Win if a winning move is available.
2. Otherwise block the opponent's winning move.
3. Otherwise take the centre, then a corner, then an edge.

Program

```
"""Experiment 1: Tic-Tac-Toe (Human vs Computer)"""

board = [' '] * 9
wins = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]

def show():
    for i in range(0, 9, 3):
        print(' ' + ' | '.join(board[i:i+3]))
        if i < 6:
            print('---+---+---')
    print()

def winner(p):
    return any(all(board[i] == p for i in w) for w in wins)

def find_move(p):
    """Return a cell where player p would win immediately."""
    for i in range(9):
        if board[i] == ' ':
            board[i] = p
            ok = winner(p)
            board[i] = ' '
            if ok:
                return i
    return None

def computer():
    m = find_move('O')              # try to win
    if m is None:
        m = find_move('X')          # block human
    if m is None:                   # centre, corners, edges
        m = [i for i in (4, 0, 2, 6, 8, 1, 3, 5, 7) if board[i] == ' '][0]
    board[m] = 'O'

def main():
    print("You are X, Computer is O. Positions are 1-9.")
    show()
    for turn in range(9):
        if turn % 2 == 0:
            while True:
```

```
                try:
                    m = int(input("Your move (1-9): ")) - 1
                    if 0 <= m < 9 and board[m] == ' ':
                        break
                except ValueError:
                    pass
                print("Invalid move, try again.")
            board[m] = 'X'
        else:
            print("Computer moves:")
            computer()
        show()
        if winner('X'):
            print("You win!"); return
        if winner('O'):
            print("Computer wins!"); return
    print("It's a draw!")

main()
```

Output

```
You are X, Computer is O. Positions are 1-9.
   |   |
---+---+---
   |   |
---+---+---
   |   |

Your move (1-9): 1
 X |   |
---+---+---
   |   |
---+---+---
   |   |

Computer moves:
 X |   |
---+---+---
   | O |
---+---+---
   |   |

Your move (1-9): 9
 X |   |
---+---+---
   | O |
---+---+---
   |   | X

Computer moves:
 X |   | O
---+---+---
   | O |
---+---+---
   |   | X

Your move (1-9): 7
 X |   | O
---+---+---
   | O |
---+---+---
```

```
 X |   | X

Computer moves:
 X |   | O
---+---+---
 O | O |
---+---+---
 X |   | X
Your move (1-9): 8
 X |   | O
---+---+---
 O | O |
---+---+---
 X | X | X
You win!
```

Conclusion

The Tic-Tac-Toe game was implemented successfully using a rule-based computer player.

Experiment 2

Language: Python 3 **Date:** _____

Problem Statement

Write a program to implement BFS (for 8 puzzle problem or Water Jug problem or any AI search problem).

Aim

To implement Breadth First Search on the Water Jug problem.

Theory

There are two jugs of 4 L and 3 L with no markings. The goal is to get exactly 2 L in either jug. A state is (a, b), the litres in each jug. The allowed operations are: fill a jug, empty a jug, and pour one jug into the other until the source is empty or the target is full.

BFS explores the state space level by level using a queue (FIFO). Because it expands all states at depth d before depth d+1, it always finds the path with the fewest steps.

Program

```
"""Experiment 2: BFS on the Water Jug problem (4L and 3L jugs, goal = 2L)"""
from collections import deque

def successors(state, cap_a, cap_b):
    a, b = state
    pour_ab = min(a, cap_b - b)
    pour_ba = min(b, cap_a - a)
    return [(cap_a, b), (a, cap_b),            # fill A, fill B
            (0, b), (a, 0),                    # empty A, empty B
            (a - pour_ab, b + pour_ab),        # pour A -> B
            (a + pour_ba, b - pour_ba)]        # pour B -> A

def bfs(cap_a, cap_b, goal):
    start = (0, 0)
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        state, path = queue.popleft()
        if goal in state:
            return path
        for nxt in successors(state, cap_a, cap_b):
            if nxt not in visited:
                visited.add(nxt)
                queue.append((nxt, path + [nxt]))
    return None

path = bfs(4, 3, 2)
print("BFS solution (4L jug, 3L jug):")
for step, s in enumerate(path):
    print("Step", step, ":", s)
print("Total steps:", len(path) - 1)
```

Output

```
BFS solution (4L jug, 3L jug):
Step 0 : (0, 0)
Step 1 : (0, 3)
Step 2 : (3, 0)
Step 3 : (3, 3)
Step 4 : (4, 2)
Total steps: 4
```

Conclusion

BFS found the shortest solution to the Water Jug problem in 4 steps.

Experiment 3

Language: Python 3 **Date:** _____

Problem Statement

Write a program to implement DFS (for 8 puzzle problem or Water Jug problem or any AI search problem)

Aim

To implement Depth First Search on the Water Jug problem.

Theory

DFS goes as deep as possible along one branch before backtracking. It is implemented with recursion (an implicit stack) and a visited set to avoid loops. DFS finds a solution, but not necessarily the shortest one.

Program

```
"""Experiment 3: DFS on the Water Jug problem (4L and 3L jugs, goal = 2L)"""

def successors(state, cap_a, cap_b):
    a, b = state
    pour_ab = min(a, cap_b - b)
    pour_ba = min(b, cap_a - a)
    return [(cap_a, b), (a, cap_b),
            (0, b), (a, 0),
            (a - pour_ab, b + pour_ab),
            (a + pour_ba, b - pour_ba)]

def dfs(state, path, visited, cap_a, cap_b, goal):
    if goal in state:
        return path
    for nxt in successors(state, cap_a, cap_b):
        if nxt not in visited:
            visited.add(nxt)
            result = dfs(nxt, path + [nxt], visited, cap_a, cap_b, goal)
            if result:
                return result
    return None

start = (0, 0)
path = dfs(start, [start], {start}, 4, 3, 2)
print("DFS solution (4L jug, 3L jug):")
for step, s in enumerate(path):
    print("Step", step, ":", s)
print("Total steps:", len(path) - 1)
```

Output

```
DFS solution (4L jug, 3L jug):
Step 0 : (0, 0)
Step 1 : (4, 0)
Step 2 : (4, 3)
Step 3 : (0, 3)
Step 4 : (3, 0)
Step 5 : (3, 3)
Step 6 : (4, 2)
Total steps: 6
```

Conclusion

DFS found a solution in 6 steps whereas BFS found one in 4 steps, so DFS does not guarantee the shortest path.

Experiment 4

Language: Python 3 **Date:** _____

Problem Statement

Write a program to implement Single Player Game (Using any Heuristic Function)

Aim

To solve the 8-puzzle (a single-player game) using Greedy Best-First Search with a heuristic function.

Theory

The heuristic h(n) is the Manhattan distance: the sum, over all tiles, of the horizontal and vertical distance of the tile from its goal position. Greedy Best-First Search always expands the state with the smallest h(n) using a priority queue. It is fast but does not guarantee the shortest solution.

Program

```
"""Experiment 4: Single-player game (8-puzzle) using Greedy Best-First Search
with the Manhattan distance heuristic."""
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def h(state):
    """Manhattan distance heuristic."""
    d = 0
    for i, tile in enumerate(state):
        if tile != 0:
            g = GOAL.index(tile)
            d += abs(i // 3 - g // 3) + abs(i % 3 - g % 3)
    return d

def neighbors(state):
    i = state.index(0)
    r, c = divmod(i, 3)
    for dr, dc, name in [(-1,0,'Up'), (1,0,'Down'), (0,-1,'Left'), (0,1,'Right')]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            j = nr * 3 + nc
            l = list(state)
            l[i], l[j] = l[j], l[i]
            yield tuple(l), name

def greedy(start):
    heap = [(h(start), 0, start, [])]
    visited = {start}
    count = 1
    while heap:
        _, _, state, moves = heapq.heappop(heap)
        if state == GOAL:
            return moves
        for nxt, name in neighbors(state):
            if nxt not in visited:
                visited.add(nxt)
                heapq.heappush(heap, (h(nxt), count, nxt, moves + [name]))
                count += 1
    return None
```

```
start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
print("Start state:", start, " h =", h(start))
moves = greedy(start)
print("Moves (blank tile):", moves)
print("Number of moves:", len(moves))
```

Output

```
Start state: (1, 2, 3, 4, 0, 6, 7, 5, 8)  h = 2
Moves (blank tile): ['Down', 'Right']
Number of moves: 2
```

Conclusion

The 8-puzzle was solved using a Manhattan distance heuristic with Greedy Best-First Search.

Experiment 5

Language: Python 3 **Date:** _____

Problem Statement

Write a program to Implement A* Algorithm.

Aim

To implement the A* search algorithm (demonstrated on the 8-puzzle).

Theory

A* uses the evaluation function f(n) = g(n) + h(n), where g(n) is the cost from the start to n (number of moves so far) and h(n) is the estimated cost from n to the goal (Manhattan distance). When h(n) never overestimates the true cost (admissible), A* is both complete and optimal.

Program

```
"""Experiment 5: A* search (shown on the 8-puzzle), f(n) = g(n) + h(n)"""
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def h(state):
    d = 0
    for i, tile in enumerate(state):
        if tile != 0:
            g = GOAL.index(tile)
            d += abs(i // 3 - g // 3) + abs(i % 3 - g % 3)
    return d

def neighbors(state):
    i = state.index(0)
    r, c = divmod(i, 3)
    for dr, dc, name in [(-1,0,'Up'), (1,0,'Down'), (0,-1,'Left'), (0,1,'Right')]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            j = nr * 3 + nc
            l = list(state)
            l[i], l[j] = l[j], l[i]
            yield tuple(l), name
def a_star(start):
    heap = [(h(start), 0, 0, start, [])]      # (f, g, tie-breaker, state, moves)
    best_g = {start: 0}
    count = 1
    while heap:
        f, g, _, state, moves = heapq.heappop(heap)
        if state == GOAL:
            return moves
        for nxt, name in neighbors(state):
            ng = g + 1
            if nxt not in best_g or ng < best_g[nxt]:
                best_g[nxt] = ng
                heapq.heappush(heap, (ng + h(nxt), ng, count, nxt, moves + [name]))
                count += 1
    return None
start = (1, 2, 3, 0, 4, 6, 7, 5, 8)
print("Start state:", start)
moves = a_star(start)
print("Optimal moves:", moves)
print("Number of moves:", len(moves))
```

Output

```
Start state: (1, 2, 3, 0, 4, 6, 7, 5, 8)
Optimal moves: ['Right', 'Down', 'Right']
Number of moves: 3
```

Conclusion

A* found the optimal solution of the 8-puzzle.

Experiment 6

Language: Python 3 **Date:** _____

Problem Statement

Write a program to implement mini-max algorithm for any game development.

Aim

To implement the Minimax algorithm for a two-player game (Tic-Tac-Toe).

Theory

Minimax is used in two-player zero-sum games. MAX (X) tries to maximise the score and MIN (O) tries to minimise it. The game tree is explored down to terminal states and the values are propagated back up.

Scores used: +10 - depth for an X win, depth - 10 for an O win, and 0 for a draw. Subtracting the depth makes quicker wins preferable.

Program

```
"""Experiment 6: Minimax algorithm (Tic-Tac-Toe). MAX = X, MIN = O."""

wins = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]

def winner(b):
    for x, y, z in wins:
        if b[x] != ' ' and b[x] == b[y] == b[z]:
            return b[x]
    return None

def minimax(b, depth, is_max):
    w = winner(b)
    if w == 'X': return 10 - depth
    if w == 'O': return depth - 10
    if ' ' not in b: return 0
    if is_max:
        best = -100
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'X'
                best = max(best, minimax(b, depth + 1, False))
                b[i] = ' '
        return best
    else:
        best = 100
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'O'
                best = min(best, minimax(b, depth + 1, True))
                b[i] = ' '
        return best
def best_move(b):
    best_score, move = -100, None
    for i in range(9):
        if b[i] == ' ':
            b[i] = 'X'
            s = minimax(b, 1, False)
            b[i] = ' '
            if s > best_score:
```

```
                best_score, move = s, i
    return move, best_score

board = ['X', 'X', ' ',
         'O', 'O', ' ',
         ' ', ' ', ' ']
print("Current board:")
for i in range(0, 9, 3):
    print(board[i:i+3])
m, s = best_move(board)
print("Best move for X: position", m + 1, "(score", s, ")")
```

Output

```
Current board:
['X', 'X', ' ']
['O', 'O', ' ']
[' ', ' ', ' ']
Best move for X: position 3 (score 9 )
```

Conclusion

The Minimax algorithm correctly chose the winning move (position 3) for X.

Experiment 7

Language: SWI-Prolog **Date:** _____

Problem Statement

Assume given a set of facts of the form father(name1,name2) (name1 is the father of name2).

Aim

To represent a family tree as Prolog facts and query it.

Theory

A Prolog program is a set of facts and rules. The fact father(X,Y) means that X is the father of Y. Queries are answered using unification and backtracking, which finds all possible answers.

Program

```
% Experiment 7: Family tree facts, father(name1, name2)
% Run:  swipl exp07_family_facts.pl
% Then try the queries below at the ?- prompt.

father(a,b).
father(a,c).
father(b,d).
father(b,e).
father(c,f).

% Sample queries:
%   ?- father(a,b).
%   ?- father(X,d).
%   ?- father(a,X).
%   ?- father(X,Y).
```

Output

```
?- father(a,b).
true.

?- father(X,d).
X = b.

?- father(a,X).
X = b ;
X = c.

?- father(X,Y).
X = a, Y = b ;
X = a, Y = c ;
X = b, Y = d ;
X = b, Y = e ;
X = c, Y = f.
```

Conclusion

Facts were stored and queried successfully in Prolog.

Experiment 8

Language: SWI-Prolog **Date:** _____

Problem Statement

Define a predicate brother(X,Y) which holds iff X and Y are brothers. Define a predicate cousin(X,Y) which holds iff X and Y are cousins. Define a predicate grandson(X,Y) which holds iff X is a grandson of Y. Define a predicate descendent(X,Y) which holds iff X is a descendent of Y. Consider the following genealogical tree: father(a,b). father(a,c). father(b,d). father(b,e). father(c,f). Say which answers, and in which order, are generated by your definitions for the following queries in Prolog:

Aim

To define the predicates brother, cousin, grandson and descendent, and show the answers generated for each query.

Theory

brother(X,Y): X and Y have the same father Z and X \= Y.
cousin(X,Y): the fathers of X and Y are brothers.
grandson(X,Y): X's father is a child of Y.
descendent(X,Y): X is a child of Y (base case), or X is a descendent of some child Z of Y (recursive case).

Answers are produced in the order of the clauses (top to bottom) and by depth-first backtracking.

Program

```
% Experiment 8: brother, cousin, grandson, descendent
% Run:  swipl exp08_relations.pl

father(a,b).
father(a,c).
father(b,d).
father(b,e).
father(c,f).
brother(X,Y)    :- father(Z,X), father(Z,Y), X \= Y.
cousin(X,Y)     :- father(A,X), father(B,Y), brother(A,B).
grandson(X,Y)   :- father(Y,Z), father(Z,X).
descendent(X,Y) :- father(Y,X).
descendent(X,Y) :- father(Y,Z), descendent(X,Z).

% Sample queries:
%   ?- brother(X,Y).
%   ?- cousin(X,Y).
%   ?- grandson(X,a).
%   ?- descendent(X,a).
```

Output

```
?- brother(X,Y).
X = b, Y = c ;
X = c, Y = b ;
X = d, Y = e ;
X = e, Y = d.

?- cousin(X,Y).
X = d, Y = f ;
X = e, Y = f ;
X = f, Y = d ;
X = f, Y = e.

?- grandson(X,a).
X = d ;
X = e ;
X = f.
?- descendent(X,a).
X = b ;
X = c ;
X = d ;
X = e ;
X = f.
```

In descendent(X,a), the direct children b and c come from clause 1, and the deeper descendents d, e and f come from the recursive clause 2.

Conclusion

All four predicates were defined and their answers (and order) were verified.

Experiment 9

Language: SWI-Prolog **Date:** _____

Problem Statement

Write a program to solve Tower of Hanoi problem using Prolog..

Aim

To solve the Tower of Hanoi problem in Prolog.

Theory

To move N disks from peg A to peg B using peg C:

1. Move N-1 disks from A to C.
2. Move disk N from A to B.
3. Move N-1 disks from C to B.

The number of moves required is 2^N - 1 (7 moves for 3 disks).

Program

```
% Experiment 9: Tower of Hanoi
% Run:  swipl exp09_hanoi.pl
%   ?- hanoi(3,left,right,center).

hanoi(0,_,_,_) :- !.
hanoi(N,From,To,Via) :-
    N > 0,
    M is N-1,
    hanoi(M,From,Via,To),
    format('Move disk ~w from ~w to ~w~n',[N,From,To]),
    hanoi(M,Via,To,From).
```

Output

```
?- hanoi(3,left,right,center).
Move disk 1 from left to right
Move disk 2 from left to center
Move disk 1 from right to center
Move disk 3 from left to right
Move disk 1 from center to left
Move disk 2 from center to right
Move disk 1 from left to right
true.
```

Conclusion

The Tower of Hanoi was solved recursively in 2^N - 1 moves.

Experiment 10

Language: SWI-Prolog **Date:** _____

Problem Statement

Write a program to solve N-Queens problem using Prolog.

Aim

To solve the N-Queens problem in Prolog.

Theory

The solution is a permutation of 1..N. Position i of the list holds the column of the queen placed in row i. Because it is a permutation, no two queens share a row or a column. The predicate no_attack checks that no two queens share a diagonal.

Program

```
% Experiment 10: N-Queens
% Run:  swipl exp10_nqueens.pl
%   ?- queens(4,Qs).

queens(N,Qs) :-
    numlist(1,N,Ns),
    permutation(Ns,Qs),
    safe(Qs).
safe([]).
safe([Q|Qs]) :- no_attack(Q,Qs,1), safe(Qs).

no_attack(_,[],_).
no_attack(Q,[Q1|Qs],D) :-
    Q =\= Q1 + D,
    Q =\= Q1 - D,
    D1 is D+1,
    no_attack(Q,Qs,D1).
```

Output

```
?- queens(4,Qs).
Qs = [2, 4, 1, 3] ;
Qs = [3, 1, 4, 2].

Board for Qs = [2,4,1,3] (row 1 at the top):
. Q . .
. . . Q
Q . . .
. . Q .
```

Conclusion

Both solutions of the 4-Queens problem were found.

Experiment 11

Language: SWI-Prolog **Date:** _____

Problem Statement

Write a program to solve 8 puzzle problem using Prolog.

Aim

To solve the 8-puzzle in Prolog.

Theory

The state is a list of 9 elements in which 0 is the blank. A move swaps the blank with an adjacent tile. Iterative deepening DFS tries depth limits 0, 1, 2, ... in turn, so the first solution found is the shortest one. States already on the current path are not revisited.

Program

```
% Experiment 11: 8-Puzzle using iterative deepening DFS
% Run:  swipl exp11_8puzzle.pl
%   ?- solve([1,2,3,4,0,6,7,5,8],Path).

goal([1,2,3,4,5,6,7,8,0]).

% adjacency of each board index (0..8)
adj(0,[1,3]).     adj(1,[0,2,4]).   adj(2,[1,5]).
adj(3,[0,4,6]).   adj(4,[1,3,5,7]). adj(5,[2,4,8]).
adj(6,[3,7]).     adj(7,[4,6,8]).   adj(8,[5,7]).

replace([_|T],0,X,[X|T]).
replace([H|T],I,X,[H|R]) :- I > 0, I1 is I-1, replace(T,I1,X,R).

move(S,S1) :-
    nth0(B,S,0),
    adj(B,L), member(T,L),
    nth0(T,S,X),
    replace(S,B,X,S2),
    replace(S2,T,0,S1).

dfs(S,P,_,P) :- goal(S).
dfs(S,P,D,Sol) :-
    D > 0,
    move(S,S1),
    \+ member(S1,P),
    D1 is D-1,
    dfs(S1,[S1|P],D1,Sol).

solve(Start,Path) :-
    between(0,31,D),
    dfs(Start,[Start],D,Rev),
    reverse(Rev,Path), !.
```

Output

```
?- solve([1,2,3,4,0,6,7,5,8],Path).
Path = [[1,2,3,4,0,6,7,5,8],
        [1,2,3,4,5,6,7,0,8],
        [1,2,3,4,5,6,7,8,0]].
```

Conclusion

The 8-puzzle was solved optimally using iterative deepening in Prolog.

Experiment 12

Language: SWI-Prolog **Date:** _____

Problem Statement

Write a program to solve travelling salesman problem using Prolog.

Aim

To solve the Travelling Salesman Problem for 4 cities in Prolog.

Theory

The salesman must visit every city exactly once and return to the starting city with minimum total distance. The program generates all permutations of the remaining cities (brute force, O(n!)), computes the cost of each tour and keeps the cheapest one.

Distances: a-b 10, a-c 15, a-d 20, b-c 35, b-d 25, c-d 30.

Program

```
% Experiment 12: Travelling Salesman Problem (brute force, 4 cities)
% Run:  swipl exp12_tsp.pl
%   ?- tsp(a,Route,Cost).

cities([a,b,c,d]).

dist(a,b,10).  dist(a,c,15).  dist(a,d,20).
dist(b,c,35).  dist(b,d,25).  dist(c,d,30).
d(X,Y,D) :- dist(X,Y,D) ; dist(Y,X,D).

tour_cost([_],0).
tour_cost([X,Y|T],C) :-
    d(X,Y,C1),
    tour_cost([Y|T],C2),
    C is C1 + C2.
tsp(Start,Best,Cost) :-
    cities(Cs),
    select(Start,Cs,Others),
    findall(C-[Start|Perm],
            ( permutation(Others,Perm),
              append([Start|Perm],[Start],Tour),
              tour_cost(Tour,C) ),
            L),
    keysort(L,[Cost-Path|_]),
    append(Path,[Start],Best).
```

Output

```
?- tsp(a,Route,Cost).
Route = [a, b, d, c, a],
Cost = 80.

Check: a to b (10) + b to d (25) + d to c (30) + c to a (15) = 80
```

Conclusion

The minimum cost tour (cost 80) was found for the 4-city problem.